

Algebraic Frame Validation and Bounded Truth-Row Resolution

A Fixed-Width Instruction ABI for Deterministic, Cross-Runtime Computation

Status: Engineering Write-Up

Scope: Computer engineering, instruction validation, deterministic runtimes, bounded resolution, lock-free receipt indexing

Reference ABI: `omi.delta_orbital_lexer.v0`

Canonical frame width: 128 bits

Canonical segment form: `uint16_t[8]`

Reference implementations: JavaScript, C99, WebAssembly

Conformance status: Cross-runtime equivalence verified by automated tests

1. Executive Summary

This system defines a fixed-width computational frame whose structural validity is determined by an algebraic error surface rather than by a parser, grammar, schema walker, or runtime-specific object model.

The canonical frame is a 128-bit word represented as eight 16-bit unsigned segments:

```
S[0] S[1] S[2] S[3] S[4] S[5] S[6] S[7]
0xLL00 0x03bf 0xNNNN 0x2bLL 0x2fLL 0xMMMM 0x039f 0xLLff
```

The frame contains three variable components:

```
LL    an 8-bit lens / row selector / projective binding value
NN    a 16-bit antecedent value
MM    a 16-bit consequent value
```

The remaining constants are structural delimiters. The frame is valid exactly when a quadratic error surface evaluates to zero:

```
Q(S) = E_var + E_const
valid ⇔ Q(S) = 0
```

After structural validation, the tuple `(LL, NN, MM)` is interpreted as one row of a bounded truth table. A resolver attempts to reach `MM` from `NN` under an `LL`-modulated 16-bit transition law. If the resolver succeeds in fewer than 15 steps, the row is accepted and may advance a shared replay cursor.

The full system therefore separates computation into three engineering stages:

1. Structural validation
2. Bounded truth-row resolution
3. Atomic receipt indexing

This separation is important. A frame may carry arbitrary antecedent and consequent values, but those values are not interpreted until the structural envelope has proven itself. This creates a deterministic, portable, low-level validation boundary suitable for JavaScript, C99, WebAssembly, SharedArrayBuffer systems, embedded systems, packet filters, and other fixed-width execution environments.

2. Design Goals

The design is built around the following engineering goals:

1. Fixed-width representation
2. Parser-free structural validation
3. Deterministic cross-runtime behavior
4. Explicit byte-order detection
5. Separation of envelope validity from payload meaning
6. Bounded truth-row resolution
7. Lock-free receipt indexing
8. Testable conformance across language targets

The system is not defined by a text grammar. The text representation is only a carrier. The canonical representation is the eight-segment `uint16_t[8]` vector.

The primary validity question is not:

Can this string be parsed?

The primary validity question is:

Does this 128-bit word lie on the zero surface $Q(S) = 0$?

3. Canonical Frame Layout

A frame consists of eight unsigned 16-bit segments:

```
S[0] = 0xLL00
S[1] = 0x03BF
S[2] = 0xNNNN
S[3] = 0x2BLL
S[4] = 0x2FLL
S[5] = 0xMMMM
S[6] = 0x039F
S[7] = 0xLLFF
```

The fields have the following roles:

Index	Field	Role	Constraint
S[0]	0xLL00	opening boundary / origin	high byte is LL ; low byte is 0x00
S[1]	0x03BF	fixed chiral delimiter	must equal 0x03BF
S[2]	0xNNNN	antecedent / input-side value	free 16-bit value
S[3]	0x2BLL	first interior lens binding	high byte is 0x2B ; low byte is LL
S[4]	0x2FLL	second interior lens binding	high byte is 0x2F ; low byte is LL
S[5]	0xMMMM	consequent / output-side value	free 16-bit value
S[6]	0x039F	fixed cardinal delimiter	must equal 0x039F
S[7]	0xLLFF	closing boundary / terminus	high byte is LL ; low byte is 0xFF

The frame uses four copies of the same LL value:

```
LL_s0 = (S[0] >> 8) & 0xFF
LL_s3 = S[3]          & 0xFF
LL_s4 = S[4]          & 0xFF
LL_s7 = (S[7] >> 8) & 0xFF
```

A structurally valid frame requires:

```
LL_s0 = LL_s3 = LL_s4 = LL_s7
```

This repeated binding gives the frame an internal orientation check. The beginning, interior anchors, and ending all have to agree on the same lens value.

4. Structural Validation by Quadratic Error Surface

The structural validator computes:

$$Q(S) = E_{\text{var}} + E_{\text{const}}$$

A frame is structurally valid if and only if:

$$Q(S) = 0$$

4.1 Variable Agreement Term

The variable agreement term checks the four `LL` appearances:

$$\begin{aligned} d03 &= LL_s0 - LL_s3 \\ d34 &= LL_s3 - LL_s4 \\ d47 &= LL_s4 - LL_s7 \\ E_{\text{var}} &= d03^2 + d34^2 + d47^2 \end{aligned}$$

Because every term is squared, `E_var = 0` exactly when all four extracted `LL` values agree.

4.2 Constant Alignment Term

The constant term checks the fixed delimiters and fixed byte positions:

$$\begin{aligned} E_{\text{const}} &= \\ & (S[0] \ \& \ 0x00FF)^2 \\ & + (S[1] - 0x03BF)^2 \\ & + ((S[3] \ \& \ 0xFF00) - 0x2B00)^2 \\ & + ((S[4] \ \& \ 0xFF00) - 0x2F00)^2 \\ & + (S[6] - 0x039F)^2 \\ & + ((S[7] \ \& \ 0x00FF) - 0x00FF)^2 \end{aligned}$$

Therefore:

$$E_{\text{const}} = 0$$

exactly when all fixed structural constants are in their canonical positions.

4.3 Conformance Rule

The structural validator is not a diagnostic bitmask. It is a numerical error surface.

```
Q(S) = 0      valid frame
Q(S) ≠ 0      malformed frame
```

A diagnostic layer may exist, but it is not the canonical validity predicate.

The canonical engineering rule is:

```
A conforming implementation MUST accept a frame if and only if Q(S) = 0.
```

5. Payload Separation

The fields `NN` and `MM` are free with respect to structural validation.

This means:

```
S[2] = NNNN
S[5] = MMMM
```

may vary over all 16-bit values without changing the structural validity of the envelope, provided the delimiters and `LL` agreement remain valid.

This is intentional.

The system separates two questions:

- ```
1. Is the frame structurally valid?
2. Does the truth row resolve?
```

At Gate 1, `NN` and `MM` are free variables.

At Gate 2, they become the antecedent and consequent of a bounded truth-row computation.

This separation prevents payload content from redefining frame authority. Payload values may travel inside the frame, but they cannot make an invalid frame valid.

---

## 6. Truth Row Extraction

Once  $Q(S) = 0$ , the frame yields one truth row:

```
row = (LL, NN, MM)
```

The packed 40-bit representation is:

```
truthRow = (LL << 32) | (NN << 16) | MM
```

Field layout:

| Bit range | Field | Width   |
|-----------|-------|---------|
| 39–32     | LL    | 8 bits  |
| 31–16     | NN    | 16 bits |
| 15–0      | MM    | 16 bits |

Conceptually:

```
LL ⊢ NN ⇒ MM
```

That is:

```
under lens LL, antecedent NN resolves to consequent MM
```

The frame does not assert that the relation is true merely by being structurally valid. It only makes the row eligible for resolution.

## 7. The 16-Bit $\Delta_C$ Transition Law

The resolver uses a 16-bit transition function:

```
 $\delta_C(x) = \text{rotr}(x, 1) \oplus \text{rotr}(x, 3) \oplus \text{rotr}(x, 2) \oplus C$
```

where:

```
rotl(x, n) = (x << n) | (x >> (16 - n))
rotr(x, n) = (x >> n) | (x << (16 - n))
```

All results are masked to 16 bits.

The canonical base constant is:

```
C = 0x5A3C
```

The strict truth-row resolver uses an `LL`-modulated constant:

```
C_LL = (LL × 0x1337) & 0xFFFF
```

Thus `LL` is not merely metadata. It selects the transition law used to resolve the row.

---

## 8. Bounded Truth-Row Resolution

The resolver has the signature:

```
fanoTruthResolver(LL, NN, MM) → steps
```

It returns:

```
0..14 if MM is reached from NN inside the bound
-1 if LL is invalid or the target is not reached
```

The resolution algorithm is:

```
if LL ∉ {1..7}:
 return -1

C = (LL × 0x1337) & 0xFFFF
state = NN
target = MM

if state == target:
 return 0

for step = 1 to 14:
```

```

state = $\delta_C(\text{state}, C)$
if state == target:
 return step

return -1

```

The strict Fano-bound domain is:

$$LL \in \{1, 2, 3, 4, 5, 6, 7\}$$

Values outside this set may still be structurally valid under  $Q(S) = 0$ , but they do not carry the strict projective incidence guarantee.

This creates a two-tier validity model:

```

Structural validity:
 $Q(S) = 0$

Resolved truth-row validity:
 $Q(S) = 0$ and $\text{fanoTruthResolver}(LL, NN, MM) \in \{0..14\}$

```

For cursor advancement, implementations may require the nonzero range:

$$1 \leq \text{steps} \leq 14$$

so that every accepted row advances the ring.

## 9. Fano Incidence Layer

The strict projective layer uses the finite projective plane  $PG(2,2)$ , commonly called the Fano plane.

It has:

```

7 points
7 lines
3 points per line
3 lines through each point

```

The implementation uses this incidence table:

| Line ID | Points    | Engineering Name        |
|---------|-----------|-------------------------|
| 1       | {1, 2, 3} | chiral-cardinal-trace   |
| 2       | {1, 4, 5} | service-bus-projection  |
| 3       | {1, 6, 7} | factorial-sweep-line    |
| 4       | {2, 4, 6} | inversion-mirror-bridge |
| 5       | {2, 5, 7} | erasure-coding-channel  |
| 6       | {3, 4, 7} | gossip-frontier-path    |
| 7       | {3, 5, 6} | anti-entropy-repair-arc |

Any two distinct lines intersect at exactly one point.

The implementation exposes a function equivalent to:

```
fanoIntersectionPoint(LL1, LL2)
```

which returns the common point for two distinct valid line IDs and returns zero for invalid or identical inputs.

This layer gives the resolver a finite coordination space. The important engineering property is not symbolic mysticism; it is bounded finite incidence.

## 10. Dual-Gate Instruction Pipeline

A conforming instruction pipeline has two gates.

### Gate 1 — Structural Envelope

Input:

```
S: uint16_t[8]
```

Compute:

```
Q(S)
```

If:

```
Q(S) ≠ 0
```

then reject:

```
GATE_1_STRUCTURAL_MALFORMATION
```

No truth-row extraction or payload interpretation is allowed before Gate 1 passes.

## Gate 2 — Truth-Row Resolution

If Gate 1 passes, extract:

```
LL = (S[0] >> 8) & 0xFF
NN = S[2]
MM = S[5]
```

Then compute:

```
steps = fanoTruthResolver(LL, NN, MM)
```

If:

```
steps = -1
```

then reject:

```
GATE_2_RESOLUTION_DIVERGENCE
```

If the resolver returns a valid step count, the frame has produced a bounded truth-row receipt.

## Accepted Receipt

The accepted receipt contains:

```
steps
LL
NN
```

```
MM
packedRow
```

with:

```
packedRow = (steps << 40) | (LL << 32) | (NN << 16) | MM
```

This is a 48-bit receipt before provenance tagging.

## 11. 64-Bit Receipt Slot

For replay indexing, receipts are stored in 64-bit slots.

The slot layout is:

| Bit range | Field      | Width   | Role                                            |
|-----------|------------|---------|-------------------------------------------------|
| 63–48     | provenance | 16 bits | source identity, root tag, or cryptographic tag |
| 47–40     | steps      | 8 bits  | resolver step count                             |
| 39–32     | LL         | 8 bits  | lens / projective row selector                  |
| 31–16     | NN         | 16 bits | antecedent                                      |
| 15–0      | MM         | 16 bits | consequent                                      |

Packed form:

```
slot =
 (provenance << 48)
| (steps << 40)
| (LL << 32)
| (NN << 16)
| MM
```

This gives the runtime a compact replay receipt:

```
who witnessed it
how far it resolved
which lens governed it
```

what antecedent entered  
what consequent closed

## 12. Atomic Cursor and Ring Indexing

The replay ring contains:

5040 slots

The cursor is stored separately from the receipt ring in its own 8-byte `SharedArrayBuffer`.

The cursor is advanced only through atomic compare-exchange:

```
loop:
 old = Atomics.load(cursor)
 new = (old + steps) mod 5040

 if Atomics.compareExchange(cursor, old, new) == old:
 write receipt at new slot
 break

 retry
```

The cursor may only be advanced by a resolved truth row.

The engineering rule is:

No valid resolution, no cursor movement.

Because valid positive step counts are bounded:

$$1 \leq \text{steps} \leq 14$$

and the ring has 5040 slots, consecutive successful CAS operations advance by a small nonzero stride. The CAS loop serializes concurrent writers without a lock. Losing workers retry against the updated cursor. The cursor buffer is disjoint from the receipt ring, so receipt writes cannot corrupt the cursor.

The resulting pipeline is:

```
validated frame
 ↓
resolved truth row
 ↓
step count
 ↓
atomic cursor advance
 ↓
receipt write
```

---

## 13. Boot Anchor

The system defines a boot anchor using the conventional boot-sector address:

```
BOOT_ADDRESS = 0x7C00
```

Mapped into the 5040-slot ring:

```
BOOT_SLOT = 0x7C00 mod 5040 = 1504
```

Canonical genesis parameters:

```
GENESIS_LL = 0x01
GENESIS_NN = 0x7C00
C = (0x01 × 0x1337) & 0xFFFF = 0x1337
GENESIS_MM = δ_0x1337(0x7C00) = 0x1434
GENESIS_STEPS = 1
```

Canonical genesis frame:

```
0100-03bf-7c00-2b01-2f01-1434-039f-01ff
```

The genesis frame is a golden accept fixture:

```
Q(S) = 0
steps = 1
```

---

## 14. Endianness and Orientation Detection

The canonical wire representation is big-endian at the segment level. Implementations must map byte streams into `uint16_t[8]` such that the segment constants match the canonical values.

The quadratic surface acts as an orientation detector.

For example, the canonical delimiter:

0x03BF

becomes:

0xBF03

under byte reversal.

The error term:

$(S[1] - 0x03BF)^2$

therefore becomes large immediately.

Likewise:

0x039F → 0x9F03  
0x2BLL → 0xLL2B  
0x2FLL → 0xLL2F

The frame is therefore self-rejecting under incorrect byte order.

This does not remove the need for careful serialization. It does provide a deterministic failure mode: incorrectly oriented frames do not silently validate.

---

## 15. Visual Order and Byte Authority

Text renderers may reorder visible characters, especially in environments involving bidirectional text, mixed scripts, or display transformations.

This ABI does not assign authority to visual order.

The canonical authority is the byte-to-segment mapping:

```
S[0], S[1], S[2], S[3], S[4], S[5], S[6], S[7]
```

A visual rendering may display the frame differently, but a conforming implementation validates only the canonical segment vector.

This is important for security. Display order must not alter execution order. The frame's authority comes from bytes, not glyphs.

---

## 16. Implementation Classes

The system is implemented in three classes:

| Class      | Language / Target | Role                                                |
|------------|-------------------|-----------------------------------------------------|
| Reference  | JavaScript        | canonical high-level implementation and test oracle |
| Portable   | C99               | low-level cross-platform implementation             |
| Executable | WebAssembly       | compiled runtime artifact                           |

The C99 and WebAssembly implementations must match the JavaScript reference for all exported operations.

Core exported operations include:

```
deltaC(x, C)
verifyOrbitLexer(S)
fanoTruthResolver(LL, NN, MM)
fanoIntersectionPoint(LL1, LL2)
generateDeltaCOrbit(start, C, out, maxlen)
makeGenesisTarget(LL, NN)
```

The key portability claim is:

```
same inputs → same outputs
```

across all conforming implementations.

---

## 17. Conformance Requirements

A conforming implementation must satisfy the following requirements.

### 17.1 Structural Equivalence

For all tested frames:

```
verifyOrbitLexer_JS(S)
=
verifyOrbitLexer_C99(S)
=
verifyOrbitLexer_WASM(S)
```

### 17.2 Resolver Equivalence

For tested triples:

```
fanoTruthResolver_JS(LL, NN, MM)
=
fanoTruthResolver_C99(LL, NN, MM)
=
fanoTruthResolver_WASM(LL, NN, MM)
```

### 17.3 Golden Accept

The genesis frame must return:

```
Q(S) = 0
steps = 1
```

### 17.4 Golden Reject — Gate 1

Malformed frames must return:

```
Q(S) ≠ 0
```

Malformed examples include:

```
wrong delimiter
wrong closure byte
```

```
wrong S3 or S4 high byte
LL disagreement
byte-swapped frame
incorrect segment count
non-hex segment in text carrier
segment overflow
```

## 17.5 Golden Reject — Gate 2

A structurally valid frame whose truth row does not resolve inside the bound must return:

```
steps = -1
```

## 17.6 Random Equivalence

Conformance tests should include randomized inputs for:

```
deltaC
fanoTruthResolver
makeGenesisTarget
fanoIntercept
fanoIntersectionPoint
verifyOrbitLexer
```

## 17.7 Period and Orbit Tests

The implementation should verify the stated transition behavior over representative values and, where practical, over the full 16-bit state space.

---

# 18. Engineering Interpretation

The system can be understood as a compact proof-carrying frame architecture.

A conventional instruction pipeline often looks like:

```
parse → interpret → execute → log
```

This system uses:

```
validate frame → resolve truth row → atomically index receipt
```

The difference is that structural authority is established before semantic interpretation.

The frame itself does not execute arbitrary behavior. It proves that it has a lawful envelope. Only then does the resolver interpret the truth row. Only a resolved row may move the cursor.

This creates a disciplined boundary between:

```
representation
validation
resolution
replay
```

That boundary is useful for systems requiring deterministic behavior across runtimes, such as:

```
language-neutral instruction encodings
WASM modules
embedded packet filters
SharedArrayBuffer coordination
browser/server equivalence tests
deterministic simulation
bootstrapping experiments
local-first distributed systems
replayable state machines
```

---

## 19. Security and Failure Properties

The ABI provides several useful failure properties.

### 19.1 Malformed Frames Fail Before Interpretation

Payload fields are not interpreted until  $Q(S) = 0$ .

### 19.2 Byte-Swapped Frames Fail Deterministically

The fixed delimiters are not byte-order symmetric.

### 19.3 Visual Reordering Does Not Create Authority

Only canonical segment order matters.

## 19.4 Invalid Projective Rows Do Not Resolve

`LL` values outside `1..7` may be structurally valid but fail strict truth-row resolution.

## 19.5 Cursor Movement Requires Resolution

The replay ring cannot advance from an unresolved truth row.

## 19.6 Concurrent Writers Are Serialized by CAS

The cursor update is lock-free and atomic.

---

# 20. Limitations and Open Engineering Questions

The implementation establishes a working ABI candidate, but several questions should remain explicit.

## 20.1 Bound Semantics

The resolver currently uses a bounded iteration limit of 14 steps. The exact mathematical proof connecting this bound to the incidence interpretation should be maintained separately from the implementation.

## 20.2 Diagnostic Layer

`Q(S)` is the canonical validity predicate. A diagnostic bitmask may be useful for tooling, but it must not replace the error-surface validator.

## 20.3 Serialization Profiles

The ABI should define exact byte serialization rules for:

```
binary carrier
hex string carrier
network packet carrier
WASM memory carrier
SharedArrayBuffer carrier
```

## 20.4 Full-State Verification

Where the ABI claims properties over all 16-bit states, the test suite should include either exhaustive verification or a documented proof.

## 20.5 Ring Overwrite Policy

The 5040-slot ring is finite. The ABI should define whether overwrite is permitted, whether receipts are epoch-tagged, and how replay consumers detect wraparound.

---

## 21. Minimal Canonical Statement

The minimal statement of the system is:

```
A frame is valid iff $Q(S) = 0$.
A valid frame yields one truth row (LL, NN, MM).
A truth row is accepted iff it resolves under δ_{C_LL} within the bounded step window.
An accepted row advances the 5040-slot replay cursor by its step count using atomic compare-exchange.
The resulting slot stores a 64-bit receipt.
```

Or, as a compact engineering summary:

```
prove the frame,
resolve the row,
advance the cursor,
store the receipt.
```

## 22. Reference Pipeline

```
byte stream
 ↓
canonical uint16_t[8] segment vector
 ↓
 $Q(S) = E_var + E_const$
 ↓
if $Q(S) \neq 0$: reject structural frame
 ↓
extract LL, NN, MM
 ↓
 $C_LL = (LL \times 0x1337) \& 0xFFFF$
 ↓
resolve $NN \rightarrow MM$ under δ_{C_LL}
 ↓
if steps = -1: reject unresolved row
```

```
 ↓
pack receipt
 ↓
CAS cursor advance mod 5040
 ↓
write 64-bit receipt
```

---

## 23. Conclusion

This ABI defines a deterministic, fixed-width computation surface in which structural validity is algebraic, payload interpretation is delayed until after validation, truth-row resolution is bounded, and accepted rows are recorded through an atomic replay cursor.

The main engineering contribution is not the specific constants alone. It is the separation of responsibilities:

```
the frame proves orientation,
the resolver proves relation,
the cursor proves replay order.
```

That separation makes the system portable, testable, and suitable for cross-runtime conformance.

The core conformance law is:

```
A frame is valid iff $Q(S) = 0$ in every conforming implementation.
```